

SESSION 3 / 8 | 4 Mayıs 2026 | 90 dakika

React Temelleri

Component, JSX, Props ve State

Session Akışı

Süre	Konu	Anahtar Kavramlar
0-20 dk	React Nedir?	SPA, Virtual DOM, declarative UI
20-45 dk	Component, JSX ve Props	function component, JSX, props
45-70 dk	State ve Event Handling	useState, onClick, onChange
70-90 dk	Pratik: Counter + Liste	useState, map, event handler

1 React Nedir?

 0–20 dakika

1.1 Neden React?

Önceki iki session'da saf HTML/CSS/JS ile arayüz oluşturduk. Küçük sayfalarda bu yeterli, ama uygulama büyüdükçe sorunlar başlar:

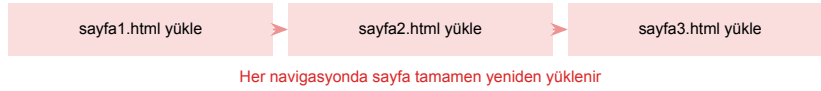
- DOM manipülasyonu karmaşıklaşır (`document.getElementById`, `innerHTML...`)
- Aynı UI parçasını tekrar tekrar yazmak gerekir
- Veri değiştiğinde hangi parçaların güncellenmesi gerektiğini takip etmek zorlaşır

React

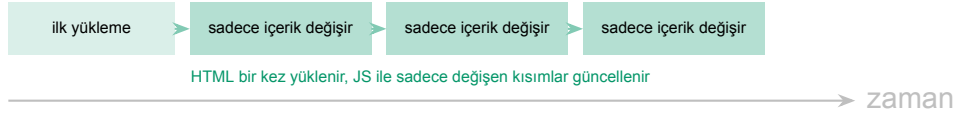
Facebook (Meta) tarafından geliştirilen, **bileşen (component) tabanlı** bir JavaScript kütüphanesidir. “UI = f(state)” felsefesine dayanır: **veri değişince arayüz otomatik güncellenir.**

1.2 SPA (Single Page Application)

Geleneksel (MPA):



SPA (React):



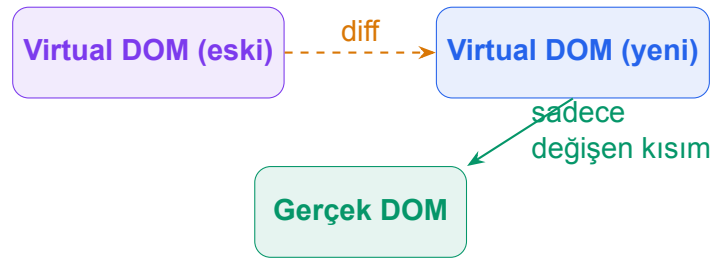
SPA Avantajları

- Daha hızlı geçişler — sayfa yeniden yüklenmez
- Daha iyi kullanıcı deneyimi — masaüstü uygulaması hissi
- Frontend ve backend bağımsız geliştirilebilir

1.3 Virtual DOM

React, gerçek DOM'u doğrudan değiştirmek yerine bellekte bir **kopya (Virtual DOM)** tutar. Veri değiştiğinde:

1. Yeni Virtual DOM oluşturulur
2. Önceki ile karşılaştırılır (**diffing**)
3. Sadece değişen kısımlar gerçek DOM'a uygulanır (**reconciliation**)

**⚠️ Önemli**

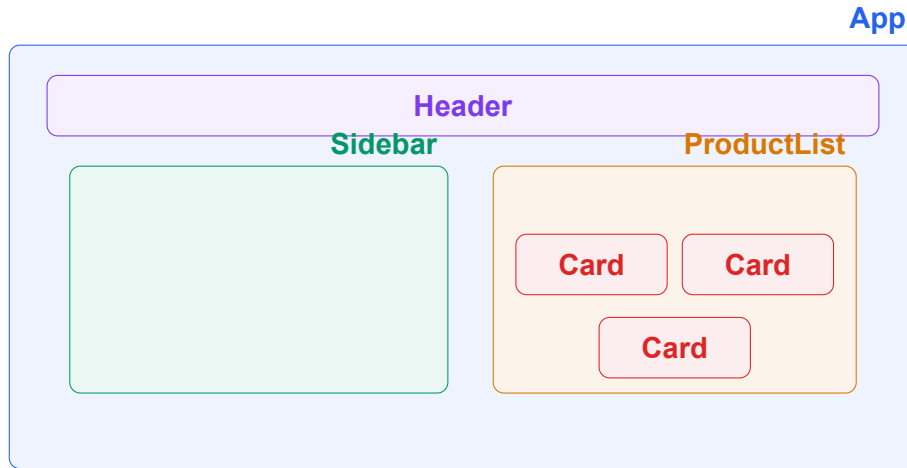
Virtual DOM “hızlı” olduğundan değil, **hangi DOM düğümlerinin değiştiğini verimli hesapladığından** faydalıdır. Tüm sayfayı yeniden çizmek yerine sadece değişen elemanları günceller.

2 Component, JSX ve Props

 20–45 dakika

2.1 Component Nedir?

React'ta her şey **component**'tir — yeniden kullanılabilir, bağımsız UI parçaları. Bir sayfa, iç içe component'lerden oluşur.



Component = Fonksiyon

React'ta bir component, **JSX döndüren bir JavaScript fonksiyonudur**. İsmi **büyük harfle** başlar.

İlk Component

```
// En basit component
function Greeting() {
  return <h1>Hello, React!</h1>;
}

// Arrow function ile (ayni sey)
const Greeting = () => {
  return <h1>Hello, React!</h1>;
};

// Kullanım
<Greeting />
```

2.2 JSX Nedir?

JSX, JavaScript içinde HTML yazmamızı sağlayan bir söz dizimidir. Tarayıcı JSX'i anlamaz; derleme araçları (Babel/SWC) onu normal JavaScript'e çevirir.

JSX vs JavaScript

```
// JSX (developer yazar)
const element = <h1 className="title">Hello</h1>;

// JavaScript (derleyici üretir)
const element = React.createElement(
  "h1",
  { className: "title" },
  "Hello"
);
```

2.2.1 JSX Kuralları

Kural	Açıklama
Tek kök eleman	Her component tek bir üst eleman döndürmeli. Birden fazla eleman varsa <div> veya <>...</> (Fragment) ile sarın.
className	HTML'deki class yerine className kullanılır (class JS'te ayrılmış kelimedir).
Süslü parantez	JS ifadeleri {...} içinde yazılır: {user.name}, {2 + 3}
Self-closing	Çocuğu olmayan etiketler kapatılmalıdır: , <input />
camelCase	HTML attribute'ları camelCase olur: onClick, onChange, htmlFor

JSX Örnekleri

```
function UserCard() {
  const name = "Ali";
  const age = 28;
  const isAdmin = true;

  return (
    <div className="card">
      /* JS ifadeleri süslü parantez içinde */
      <h2>{name}</h2>
      <p>Age: {age}</p>
      <p>Role: {isAdmin ? "Admin" : "User"}</p>
    </div>
  );
}
```

⚠ Fragment

Birden fazla eleman döndürmek istiyorsanız, gereksiz <div> eklemek yerine **Fragment** kullanın:

```
<> ... </> veya <React.Fragment> ... </React.Fragment>
```

2.3 Props — Component'e Veri Aktarımı

Props (properties), bir component'e **dışarıdan veri göndermek** için kullanılır. HTML attribute'larına benzer, ama herhangi bir JavaScript değeri olabilir.

Props Kullanımı

```
// Component tanımlarken props parametresi alınır
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

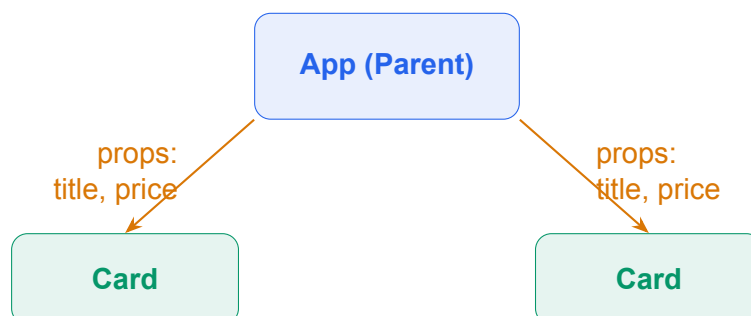
// Daha yaygın: destructuring ile
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}

// Kullanım --- attribute gibi yazılır
<Greeting name="Ali" />
<Greeting name="Zeynep" />
<Greeting name="Can" />
```

Birden Fazla Prop

```
function ProductCard({ title, price, inStock }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <p>Price: {price} TL</p>
      <p>{inStock ? "In stock" : "Out of stock"}</p>
    </div>
  );
}

// Kullanım
<ProductCard title="Notebook" price={45} inStock={true} />
<ProductCard title="Pencil" price={5} inStock={false} />
```



❗ Props Read-Only'dir

Props **değiştirilemez**. Bir component aldığı prop'u sadece **okuyabilir**. Değişebilir veri için **state** kullanılır (bir sonraki bölüm).

2.3.1 children Prop'u

Component'in açılış ve kapanış etiketi arasına yazılan içerik, otomatik olarak `children` prop'u ile gelir.

children Örneği

```
function Card({ children }) {  
  return (  
    <div className="card">  
      {children}  
    </div>  
  );  
}  
  
// Kullanım  
<Card>  
  <h2>Product Title</h2>  
  <p>Description text here.</p>  
</Card>
```

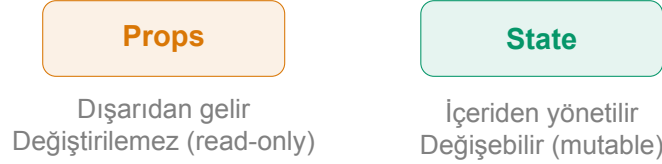
3 State ve Event Handling

 45–70 dakika

3.1 State Nedir?

Props dışarıdan gelir ve değiştirilemez. **State** ise component'in **kendi içinde tuttuğu değişebilir veridir**.

State değiştiğinde React o component'i **otomatik olarak yeniden render** eder.



3.2 useState Hook

useState, React'in en temel Hook'udur. Component'e **değişebilir veri** (state) eklememizi sağlar.

useState Formülü

```
const [deger, setDeger] = useState(baslangicDeger);
```

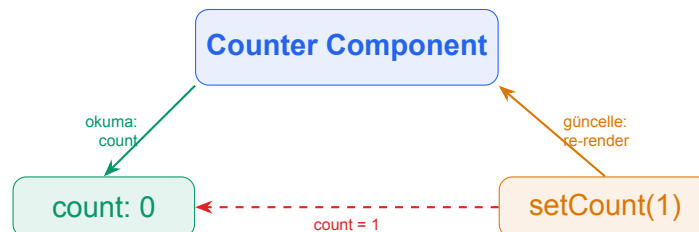
- deger — state'in şu anki değeri
- setDeger — değeri güncelleyen fonksiyon
- baslangicDeger — ilk render'daki değer

useState — Basit Counter

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}
```



3.3 Event Handling

React'ta event'ler (olaylar), HTML'deki gibi attribute olarak yazılır. Fark: **camelCase** kullanılır ve fonksiyon referansı verilir.

HTML	React (JSX)	Açıklama
onclick	onClick	Tıklama
onChange	onChange	Input değişimi
onsubmit	onSubmit	Form gönderimi
onmouseover	onMouseOver	Mouse üzerine gelme
onfocus	onFocus	Elemana odaklanma

Event Handler Örnekleri

```
function Demo() {
  // 1. Inline arrow function
  <button onClick={() => alert("Clicked!")}>
    Click Me
  </button>

  // 2. Separate function (recommended for readability)
  const handleClick = () => {
    console.log("Button clicked!");
  };
  <button onClick={handleClick}>Click Me</button>

  // 3. Event object
  const handleChange = (event) => {
    console.log("Typed:", event.target.value);
  };
  <input onChange={handleChange} />
}
```

⚠ Sık Yapılan Hata

Event handler'a fonksiyon **referansı** verilir, **çağrılmaz**:

onClick={handleClick} ✓ Doğru

onClick={handleClick()} ✗ Yanlış — render sırasında hemen çalışır!

3.4 State ile Input Kontrolü

React'ta form input'ları genellikle **controlled component** olarak kullanılır: input'un değeri state'te tutulur, her değişiklik onChange ile state'e yazılır.

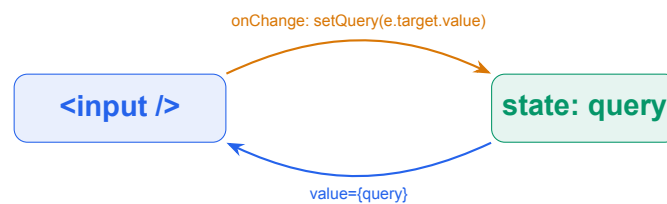
Controlled Input

```
import { useState } from "react";

function SearchBox() {
  const [query, setQuery] = useState("");

  const handleChange = (e) => {
    setQuery(e.target.value);
  };

  return (
    <div>
      <input
        type="text"
        value={query}
        onChange={handleChange}
        placeholder="Search..."
      />
      <p>Searching for: {query}</p>
    </div>
  );
}
```



💡 Neden Controlled?

State tek kaynak (single source of truth) olur. Input'un değerini her an bilebilir, filtreleyebilir veya doğrulayabiliriz. React ekosisteminde standart yaklaşım budur.

4 Pratik: Counter + Listeye Ekleme

 70–90 dakika

Pratik Hedef

İki mini uygulama oluşturacağız:

1. **Counter:** Artır / Azalt / Sıfırla butonları olan bir sayaç
2. **Listeye Ekleme:** Input'tan metin alıp listeye ekleyen uygulama

4.1 Pratik 1 — Counter Component

Counter.jsx

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div style={{ textAlign: "center", marginTop: 40 }}>
      <h1>Counter: {count}</h1>

      <button onClick={() => setCount(count - 1)}>
        - Decrease
      </button>

      <button onClick={() => setCount(0)}>
        Reset
      </button>

      <button onClick={() => setCount(count + 1)}>
        + Increase
      </button>
    </div>
  );
}

export default Counter;
```

4.2 Pratik 2 — Input'tan Listeye Ekleme

ItemList.jsx

```
import { useState } from "react";

function ItemList() {
  const [items, setItems] = useState([]);
  const [text, setText] = useState("");

  const handleAdd = () => {
    if (text.trim() === "") return;

    setItems([...items, text]);
    setText("");
  };

  return (
    <div style={{ maxWidth: 400, margin: "40px auto" }}>
      <h1>My List</h1>

      <div>
        <input
          type="text"
          value={text}
          onChange={e => setText(e.target.value)}
          placeholder="Add an item..."
        />
        <button onClick={handleAdd}>Add</button>
      </div>

      <ul>
        {items.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>

      <p>{items.length} items in list</p>
    </div>
  );
}

export default ItemList;
```

💡 Neler Kullandık?

- useState ile iki state: dizi (items) ve metin (text)
- spread ile immutable güncelleme: [...items, text]
- map ile liste render
- onChange ile controlled input

- `onClick` ile buton event'i

⚠ key Prop'u

`map` ile liste render ederken her elemana **benzersiz** `key` prop'u verilmelidir. React, hangi elemanın değiştiğini bulmak için `key` değerini kullanır.

İdeal olarak `key` için veritabanındaki `id` kullanılır. `index` son çare olarak kullanılabilir ama sıralama değişirse sorun yaratabilir.

5 Özet ve Sonraki Session

5.1 Bu Session'da Öğrendiklerimiz

- ✓ React nedir ve neden kullanılır
- ✓ SPA (Single Page Application) mantığı
- ✓ Virtual DOM ve React'in güncelleme mekanizması
- ✓ Component = JSX döndüren fonksiyon
- ✓ JSX kuralları: tek kök eleman, className, süslü parantez
- ✓ Props ile component'e veri aktarımı
- ✓ children prop'u
- ✓ useState Hook ile state yönetimi
- ✓ Event handling: onClick, onChange
- ✓ Controlled input patterni
- ✓ Pratik: Counter ve listeye eleman ekleme

5.2 Sıkça Sorulan Sorular

SSS

S: Component isimleri neden büyük harfle başlar?

C: React, küçük harfle başlayan tag'leri HTML elemanı olarak yorumlar (`<div>`, `<p>`). Büyük harfle başlayanlar ise özel component olarak tanımlanır (`<Counter />`).

S: State'i neden doğrudan değiştirmiyoruz?

C: `count = 5` yazsak React bundan haberdar olmaz ve ekranı güncellemez. `setCount(5)` dediğimizde React yeni değeri alır ve component'i yeniden render eder.

S: Her component'in kendi state'i mi var?

C: Evet. Aynı component'ten birden fazla kullanılsa bile her birinin state'i bağımsızdır. `<Counter />` iki kez yazılırsa ikisi birbirini etkilemez.

5.3 Sonraki Session — Session 4: React Derinleşme

→ Session 4 (6 Mayıs Çarşamba)

React bilgimizi proje seviyesine çıkarmak için:

- **useEffect** — yan etkiler ve lifecycle
- **Conditional rendering** — koşullu gösterim
- **List rendering** ve key konusu (detaylı)
- **Form handling** — controlled input, validation
- **Pratik:** Mini ToDo uygulaması